Московский Физико–Технический Институт

Численные методы
Задание №15

ФИО: Лазарь Владислав Игоревич
Группа: Б01-207а
Почта: lazar.v@phystech.edu

Долгопрудный, 2026 г.

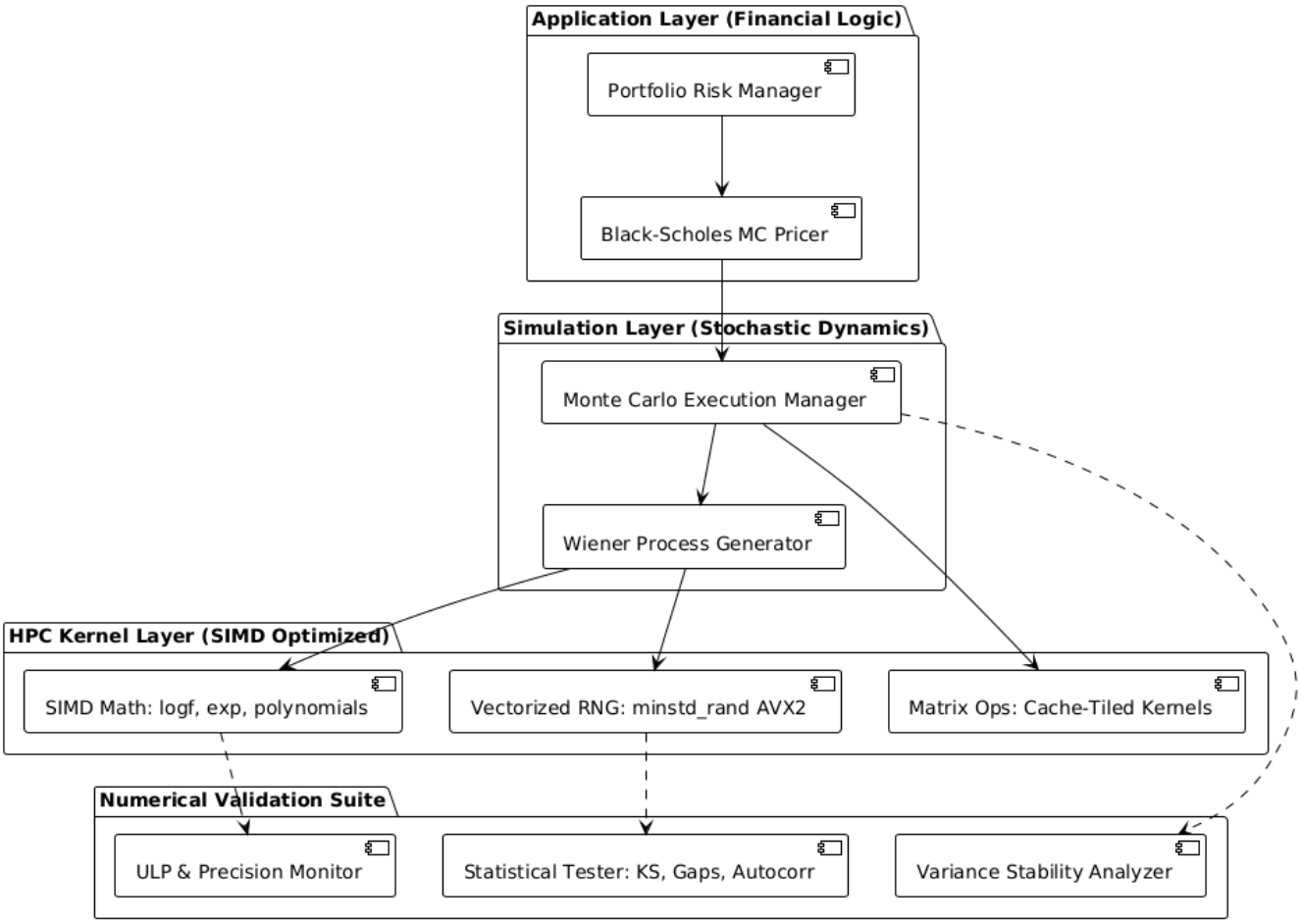
Отчет об архитектуре системы V-Quant Engine

Целевая аудитория: системные архитекторы, разработчики HPC-ядер, инженеры по вычислительной математике.
Статус: концептуальное описание (High-Level Design).

1. Описание системы

`V-Quant Engine` — это модульная экосистема для высокоточного моделирования стохастических процессов и оценки производных финансовых инструментов. Архитектура построена на принципе `Performance-First Design`, где каждый уровень (от базовой арифметики до бизнес-логики) проходит автоматизированную верификацию точности и производительности.

2. Структура системы



3. Описание архитектурных уровней

- **HPC Kernel Layer (SIMD-оптимизированное ядро)**

  Фундамент системы реализован на `C++` с использованием интринсиков (`AVX2`/`AVX512`).

  Целевой метрикой выбрана `throughput`. Было принято решение отказаться от стандартной `libm`. Необходимо добиться 3.5 `CPE` при сохранении точности $\leqslant 4$ `ULP`.

  Слой реализует векторизованную линейную алгебру (`cache tiling` для матриц $\leqslant$ L1), детерминированные ГПСЧ (`minstd_rand` с поддержкой `jump-ahead`), и аппроксимированные трансцендентные функции.

- **Numerical Validation Suite (модуль контроля качества)**

  В данном слое реализован инструментарий, гарантирующий корректность численных методов. Здесь тестируются инварианты, анализируются битовые последовательности и соответствие эмпирическим заданным распределениям. Анализируется статистический контроль ГПСЧ (KS-тест, автокорреляция, спектральный анализ).

  На этом уровне также происходит мониторинг катастрофических потерь точности при вычислении в `f32`/`f64`.

- **Simulation Layer (слой стохастики)**

  Этот слой является инфраструктурой для моделирования динамики активов. Поддерживает генерацию траекторий Винеровских процессов с малым шагом. Абстрагирует алгоритмы Эйлера–Маруямы, методы Монте–Карло. Обеспечивает распараллеливание потоков без `race-condition` через `thread-local` состояния RNG.

- **Application Layer (прикладные финансы)**

  Данный уровень абстрагирует финансовую математику от вычислительной реализации, обеспечивая разделение зон ответственности между разработчиком модели и системным программистом.

  Здесь находится главная логика оценки производных (Black-Scholes MC, расчет `VaR`/`CVaR`). Зависит исключительно от контрактов нижних слоев, не содержит платформо-зависимого кода.

4. Обоснование архитектурных решений

Обоснования в основном строятся на принципах `SOLID`.

- **Декомпозиция и наличие интерфейсов**:

  Так соблюдаются принципы **DIP** и **ISP**. Высокоуровневые модули, например, финансы или стохастика, зависят исключительно от абстрактных контрактов (`IRngStream`, `IMatrixKernel`), а не от конкретных `SIMD`-реализаций. Это же и обеспечивает минимальное знание частей друг о друге, что позволяет подменять вычислительные бэкенды (`CPU`/`GPU`/`AVX512`) без рефакторинга бизнес-логики.

- **Отказ от `libm` в пользу минимаксных аппроксимаций**:

  Здесь соблюден и **OCP**. Кастомные функции, аналогичные реализованной `logf_avx2`, полностью замещают стандартные вызовы, сохраняя семантику контракта и гарантируя точность < 3.5 ULP. Архитектура открыта для расширения новыми схемами аппроксимации без изменения вызывающего кода.

- **Абстракция генерации случайных чисел через `IRngStream`**: Слой стохастического моделирования зависит исключительно от интерфейса `IRngStream`, а не от конкретных реализаций ГПСЧ (**DIP**). Это позволяет расширять систему новыми генераторами (`minstd_rand`, `PCG64`, векторизованные версии) или стратегиями распараллеливания (`skip-ahead`, `discard`) без модификации вызывающего кода (**OCP**).

  Любая реализация, удовлетворяющая контракту `IRngStream`, гарантирует статистическую корректность и воспроизводимость. Валидация выполнена двухуровневым тестированием (KS-тест + автокорреляция), как рекомендовано в методологии верификации.

- **Cache-tiling для матричного умножения**:

  Алгоритм декомпозиции памяти вынесен в независимый вычислительный блок (**SRP**) для адаптации стратегии разбиения под разные иерархии кэшей (`L1`/`L2`) без изменения ядра симуляций. Решение следует **OCP** — добавление новых микро-ядер или изменение размера блоков не требует модификации интерфейсов `IMatrixKernel`, сохраняя производительность при изменении входных размеров.

5. Адаптация под другие архитектуры (x86, ARM, ...)

Для обеспечения переносимости между архитектурами внедрен компонент `Dispatcher`. На этапе инициализации система определяет доступные наборы инструкций (`CPUID`). При инициализации ядра автоматически выбирается оптимальный вычислительный бэкенд.

Все архитектурные изменения строго локализованы в `HPC Kernel Layer` и прозрачны для верхних слоев.